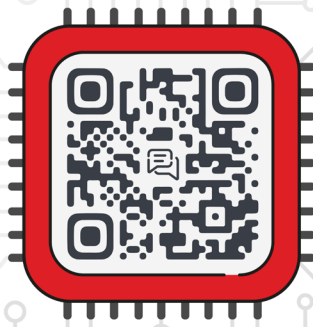




8



6



900 sp

نظري

كلية الهندسة المعلوماتية

السنة الأولى

العودية Recursion

د. مدحت الصوص

د. فراس ضعيف



برمجة ا

RB Informatics; 11/1/2026

محتوى مجاني غير مخصص للبيع التجاري

مقدمة:

يمكن لأي تابع في لغة C++ استدعاء أي تابع آخر كما يمكنه استدعاء نفسه. وعندما يقوم التابع باستدعاء نفسه نطلق عليه اسم التابع العودي. تستخدم التوابع العودية لحل المسائل التي يمكن تقسيمها إلى نفس المشكلة ولكن بشكل أبسط ومتكرر، حيث يتم حل المشكلة الأساسية من خلال حل نسخة أصغر منها، ثم استخدام هذا الحل للوصول إلى الحل النهائي. مثال على شكل التابع العودي

```
void f() {
    مجموعة تعليمات
    f();
    مجموعة تعليمات
}
```

في المثال السابق يعد التابع f تابعاً عودياً لأنه قام باستدعاء نفسه لكن كتابة تابع يستدعي نفسه فقط لا تكفي إذ يجب أن يحتوي التابع العودي على شرط يحدد متى يتوقف عن الاستدعاء

هل للتوابع العودية نهاية؟

نعم، يجب أن يكون لكل تابع عودي شرط توقف (Base Case). وهو شرط عند تحققه يتوقف التابع عن استدعاء نفسه في حال عدم وجود شرط توقف سيستمر التابع في استدعاء نفسه بشكل لا نهائي، مما يؤدي إلى حدوث خطأ يسمى: Stack Overflow ويحدث هذا الخطأ لأن كل استدعاء للتابع يحجز مساحة جديدة في الذاكرة من نوع stack لتخزين متحولاته. ومع تكرار الاستدعاء دون توقف تمتلئ الذاكرة المخصصة للتوابع، وهي ذاكرة محدودة الحجم، وعند امتلائها يتوقف البرنامج ويظهر هذا الخطأ.

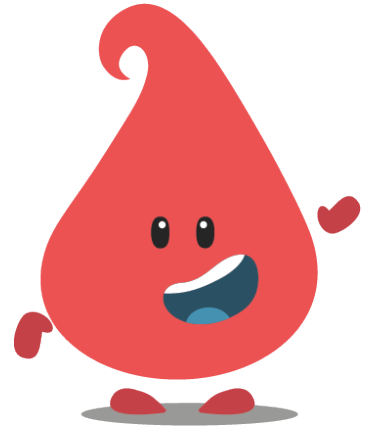
آلية عمل التوابع العودية

تعتمد فكرة العودية على مبدأ تقسيم المشكلة إلى مشكلة أبسط منها مرتبطة بها. ولحل المشكلة الأساسية نقوم بحل نسخة أصغر منها، ثم نستخدم نتيجة هذه النسخة للوصول إلى الحل النهائي. لكن حتى تنجح هذه العملية يجب الوصول في النهاية إلى حالة بسيطة جداً نعرف حلها مباشرة دون الحاجة إلى استدعاء جديد، وتسمى هذه الحالة: الحالة الأساسية (شرط التوقف). وبمجرد الوصول إلى هذه الحالة يبدأ البرنامج بالعودة تدريجياً حتى نحصل على الحل النهائي للمشكلة الأساسية.

1. حساب العامل

1. باستخدام حلقة for دون استخدام العودية

```
#include <iostream>
using namespace std;
int fact(int n)
{
    int factorial = 1;
    for (int i = 1; i <= n; i++)
    {
        factorial *= i;
    }
    return factorial;
}
int main()
{
    int x;
    cout << "Factorial calculator" << endl;
    cout << "Enter your number: ";
    cin >> x;
    cout << x << "!=" << fact(x);
    return 0;
}
```



في هذا المثال يتم حساب العامل بضرب الأعداد من 1 حتى العدد المطلوب باستخدام حلقة التكرار

2. باستخدام العودية

طريقة التفكير في الحل العودي:

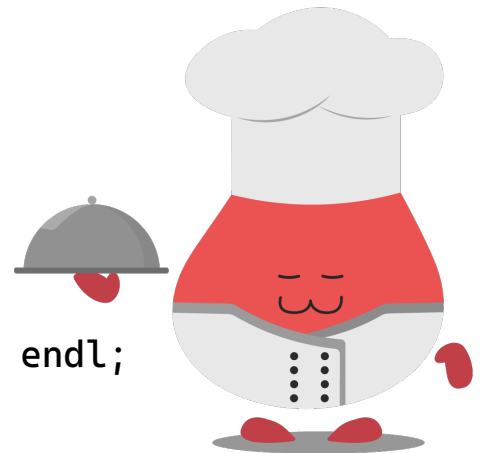
عند حل أي مسألة باستخدام العودية نطرح على أنفسنا ثلاثة أسئلة أساسية:

- 1- متى يتوقف التنفيذ؟ وهو شرط التوقف (الحالة الأبسط للمسألة)
- 2- ما هي المسألة الأبسط من النوع نفسه؟ أي كيف يمكن اختصار المسألة إلى نسخة أصغر منها
- 3- كيف نربط الحل الأبسط بالمسألة الأساسية؟ أي كيف نستخدم نتيجة المسألة الأصغر لحل المسألة الأصلية

تطبيق هذه الأسئلة على حساب العامل:

- 1- شرط التوقف: يتوقف التنفيذ عندما $n=1$ أو $n=0$
- 2- المسألة الأبسط: عند حساب $n!$ فإن المسألة الأبسط من النوع نفسه هي $(n-1)!$
- 3- تركيب الحل: يمكن كتابة العامل بالشكل: $n \times (n-1)!$

```
1. #include <iostream>
2. using namespace std;
3. int fact(int n) {
4.     if (n == 1 || n == 0)
5.         return 1;
6.     else
7.         return n * fact(n - 1);
8. }
9. int main() {
10.    int x;
11.    cout << "Factorial calculator" << endl;
12.    cout << "Enter your number: ";
13.    cin >> x;
14.    cout << x << "!=" << fact(x);
15.    return 0;
16. }
```



■ شرح الكود

في سطر if قمنا باعتبار أن $0! = 1$ و $1! = 1$ وتمثل هذه الحالة شرط التوقف لأنها أبسط حالة يمكن الوصول إليها في حال عدم تحقق شرط التوقف ينتقل التنفيذ إلى السطر `return n*fact(n-1)` حيث يقوم التابع باستدعاء نفسه لحساب العامل للعدد الأصغر $n-1$ ثم ضرب الناتج بالعدد n ويمكن الاستغناء عن كتابة `else` لأن التابع سيعود مباشرة عند تحقق شرط `if`

■ تتبع تنفيذ التابع العودي

لنفرض أننا استدعينا `fact(5)` يتم التنفيذ على الشكل التالي:

$$\begin{aligned} & fact(5) \\ &= 5 \times fact(4) \\ &= 5 \times 4 \times fact(3) \\ &= 5 \times 4 \times 3 \times fact(2) \\ &= 5 \times 4 \times 3 \times 2 \times fact(1) \\ &= 5 \times 4 \times 3 \times 2 \times 1 \end{aligned}$$

وعند الوصول إلى `fact(1)=1` يتوقف الاستدعاء ويبدأ البرنامج بالعودة تدريجياً إلى الأعلى لحساب النتيجة النهائية فتكون النتيجة `fact(5)=120`

في التوابع العودية

يبدأ التنفيذ بالنزول نحو الحالة الأبسط (شرط التوقف)

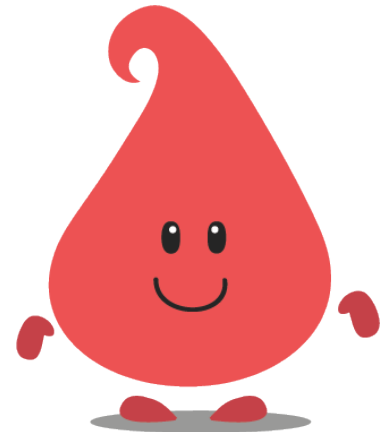
ثم يبدأ حساب النتيجة والعودة من الأسفل إلى الأعلى حتى الوصول إلى الاستدعاء الأول

2. تابع عودي لطباعة الأعداد من 1 إلى n

لحل المسألة باستخدام العودية نجيب على الأسئلة الثلاثة الأساسية

- 1- متى يتوقف التابع: يتوقف التنفيذ عند $n=1$ وتمثل هذه الحالة أبسط حالة في المسألة
- 2- ما هي الحالة الأبسط من النوع نفسه: لطباعة الأعداد حتى n نحتاج أولاً إلى طباعة الأعداد حتى $n-1$
- 3- كيف نركب الحل: نطبع الأعداد حتى $n-1$ أولاً ثم نطبع العدد n

```
#include <iostream>
using namespace std;
void print(int n) {
    if (n == 1)
        cout << n;
    else {
        print(n - 1);
        cout << n;
    }
}
int main()
{
    int n;
    cin >> n;
    print(n);
}
```



■ شرح النقاط المهمة في الكود:

عند استدعاء التابع `print(n)` يقوم التابع أولاً باستدعاء نفسه بالقيمة الأصغر ($n-1$)، ويستمر ذلك حتى يصل إلى القيمة 1، وهي أبسط حالة في المسألة. عندها يبدأ التنفيذ بالعودة تدريجياً من الاستدعاءات السابقة. أثناء العودة يتم تنفيذ جملة الطباعة: `cout << n` ولأن هذه الجملة موضوعة بعد الاستدعاء العودي، فإن الطباعة لا تتم أثناء النزول نحو الحالة الأبسط، بل تتم أثناء العودة من الاستدعاءات. لذلك تظهر الأعداد بالترتيب من 1 حتى n . فعند استدعاء: `print(5)` يكون مسار التنفيذ على الشكل:

`print(4) → print(3) → print(2) → print(1)`

ثم يبدأ البرنامج بالعودة للأعلى وتنفيذ الطباعة، فتكون النتيجة:

1 2 3 4 5



3. تابع عودي لطباعة الأعداد من n إلى 1

```
#include <iostream>
using namespace std;

void print(int n)
{
    if (n == 1)
        cout << n;
    else {
        cout << n;
        print(n - 1);
    }
}

int main()
{
    int n;
    cin >> n;
    print(n);
}
```



نستنتج من المثال الثاني والثالث:

في التتابع العودية يعتمد ترتيب التنفيذ على موقع التعليلة بالنسبة للاستدعاء العودي:
 من 1 إلى n في حال كانت الطباعة بعد الاستدعاء (كما في المثال الثاني)
 من n إلى 1 في حال كانت الطباعة قبل الاستدعاء (كما في المثال الثالث)

4. تابع عودي لإيجاد القاسم المشترك الأكبر GCD

لحساب القاسم المشترك الأكبر سنستخدم خوارزمية إقليدس، والتي تعتمد على طرح العدد الأصغر من العدد الأكبر بشكل متكرر حتى يبع العددين متساويين، وعندها يكون هذا العدد هو القاسم المشترك الأكبر.
 أي أننا في كل خطوة نستبدل العدد الأكبر بفرق العددين، ونكرر العملية حتى الوصول إلى حالتين متساويتين
 مثال توضيحي: حساب القاسم المشترك الأكبر للعددين 21 و 12

$$\begin{aligned} GCD(21,12) &= 21 - 12 = 9 \\ GCD(9,12) &= 12 - 9 = 3 \\ GCD(9,3) &= 9 - 3 = 6 \\ GCD(6,3) &= 6 - 3 = 3 \\ GCD(3,3) &= 3 - 3 = 0 \end{aligned}$$

ومنه نستنتج أن القاسم المشترك الأكبر للعددين 21 و 12 هو 3

مثال: اكتب برنامج لحساب القاسم المشترك الأكبر لعددين باستخدام العودية

```
#include <iostream>
using namespace std;

int GCD(int x, int y) {
    if (x == y)
        return x;
    else {
        if (x > y)
            return GCD(x - y, y);
        else
            return GCD(x, y - x);
    }
}

int main() {
    int a, b;
    cin >> a >> b;
    cout << GCD(a, b);
}
```



شرح عمل التابع

يعتمد التابع على مقارنة العددين في كل استدعاء:

- إذا أصبح العددين متساويين فإن القاسم المشترك الأكبر هو هذا العدد نفسه، لذلك اعتبرت هذه الحالة شرط التوقف.
 - إذا كان العدد الأول أكبر من الثاني نقوم بطرح الثاني من الأول ثم نعيد استدعاء التابع.
 - إذا كان العدد الثاني أكبر نقوم بطرح الأول من الثاني ثم نعيد الاستدعاء.
- تستمر العملية في كل مرة بقيم أصغر حتى يصبح العددين متساويين، وعندها يبدأ التابع بإعادة القيمة عبر جميع الاستدعاءات حتى تصل النتيجة النهائية إلى الاستدعاء الأول.
- ويجب استخدام return في جميع الحالات لأن التابع يعيد قيمة من النوع int، ولأن نتيجة كل استدعاء يجب إرجاعها للاستدعاء الذي سبقه حتى نحصل على الناتج النهائي

